PROJECT REPORT / REVIEW DRAFT

EMBER

Worth the craving.

Restaurant Ordering Frontend

CSCI426 - Advanced Web Programming
Project Phase 1



Title-page field	Required detail
Student	Jaafar Sabra
Student ID	72230515
Instructor	Ghadir Khalil
Institution	Lebanese International University (LIU)
Project mode	Individual project
Repository / approved live URL	Pending GitHub and Vercel publication

Draft status: technical content and screenshots reflect the working frontend. Student details are complete. Add the final links and phone screenshots and finish mobile checks before submission.

01 / ABSTRACT AND SCOPE

A complete frontend ordering journey.

EMBER is a React-based restaurant ordering application that helps customers browse a varied menu, inspect dishes, customize a meal, manage a bag, choose delivery or pickup and view order progress. The design uses charcoal, warm ivory and burnt orange with consistent typography and locally served food imagery.

Phase 1 focuses on frontend behavior and responsive UI. React state controls interactions, while localStorage preserves the cart and most recent order. Tracking is simulated from an order timestamp; no real payment, email or kitchen integration is claimed.

Technologies used

React 19 and JavaScript (JSX) provide reusable components and state. Vite 6 builds the site. HTML5 and CSS provide structure, responsive layouts and transitions. Browser localStorage persists local data; Git records development. Google Maps and Google Fonts support the contact map and typography.

Assignment coverage

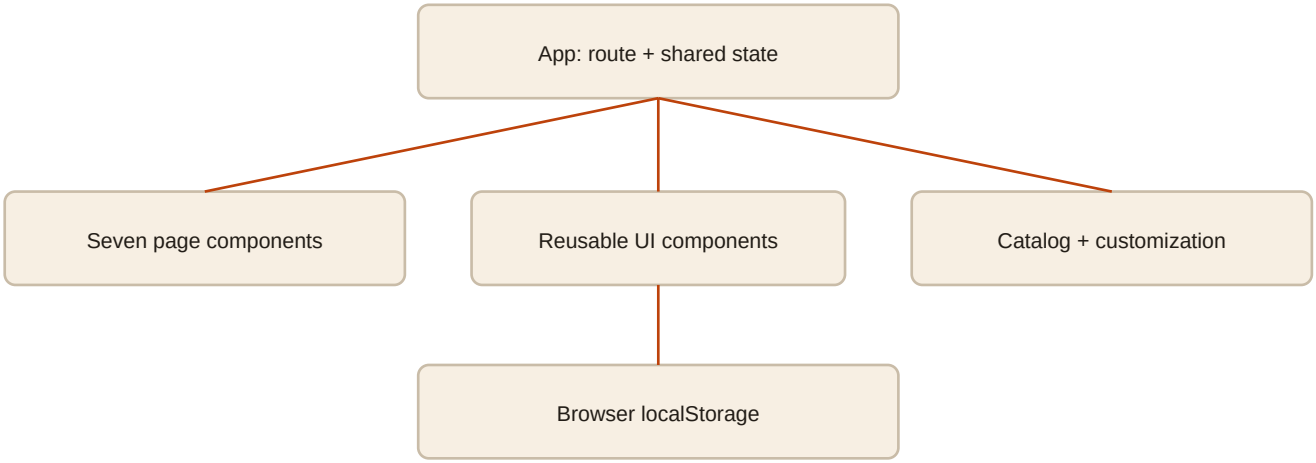
Requirement	Implemented evidence
React frontend	React 19 with Vite 6; functional components and hooks.
At least five pages	Home, Our Story, Experience, Menu, Contact, Checkout and Tracking.
Dynamic page	Search/filter results, product options, bag totals and tracking change with state.
Restaurant theme	Menu display, cart and order tracking.
Responsive design	Desktop, tablet and mobile CSS breakpoints; collapsible mobile navigation. Physical-device checks remain to finish.
Git / GitHub / approved host	Genuine local development history exists. Student GitHub publication and approved hosting remain pending.
README	Description, setup, architecture, attribution and eight student-supplied UI screenshots included.

Team and integrity

This is an individual project submitted by Jaafar Sabra (72230515). The student must describe actual work and disclose AI assistance and image sources according to course policy. No pre-built website template was used.

02 / SYSTEM DESIGN

Small components. Shared state.



The App component owns the selected route, bag, latest order, customization choices and dialogs. Page components render the route-specific layout. Shared UI includes icons, navigation elements, modal focus management, the photo viewer, animated product flight and contact map.

Source	Responsibility
src/main.jsx	Mount React and load the stylesheet.
src/App.jsx	Hash navigation, cart updates, customization state and order creation.
src/pages/	Seven separate page components.
src/components/UI.jsx	Reusable controls, gallery, modal, hero, map and motion.
src/data/menu.js	Product catalog, image paths, price formatting and safe storage reads.
src/data/customization.js	Category-specific choices, extras and preferences.
style.css / public/assets/	Responsive design, transitions and locally served images.

Data and event flow

Menu selection opens customization. Paid choices and selected extras are added to the base price. Quantity multiplies the unit price. A cart key includes the product, selection summary and note, preventing different customizations from being merged. Checkout saves an order snapshot before clearing the bag.

Persistence and routing

The ember-cart, ember-order and ember-contact localStorage keys retain browser-local data. They are not a database and are not shared between customers or devices. Hash-based URLs select page components without server route rewrites.

03 / INTERFACE

Menu and customization.

The following screenshots were supplied by Jaafar Sabra from the working application on 8 November 2026. They replace the earlier development captures.

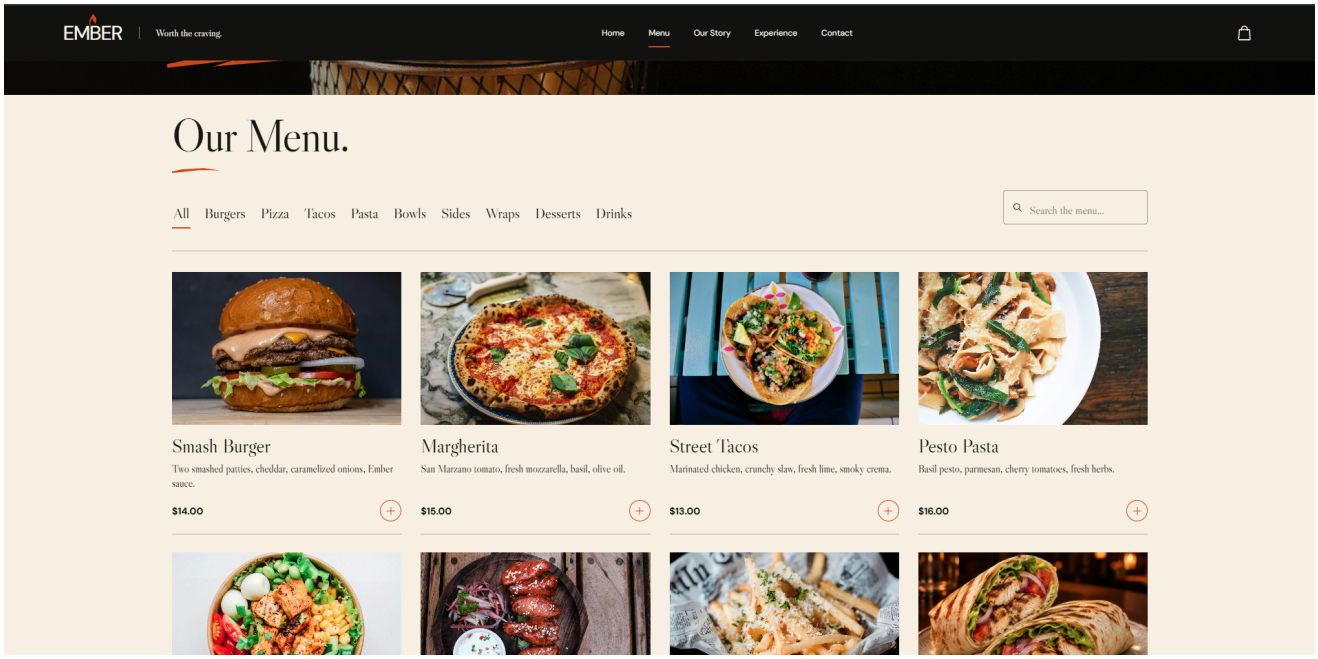


Figure 1. Menu: categories, search and product cards.

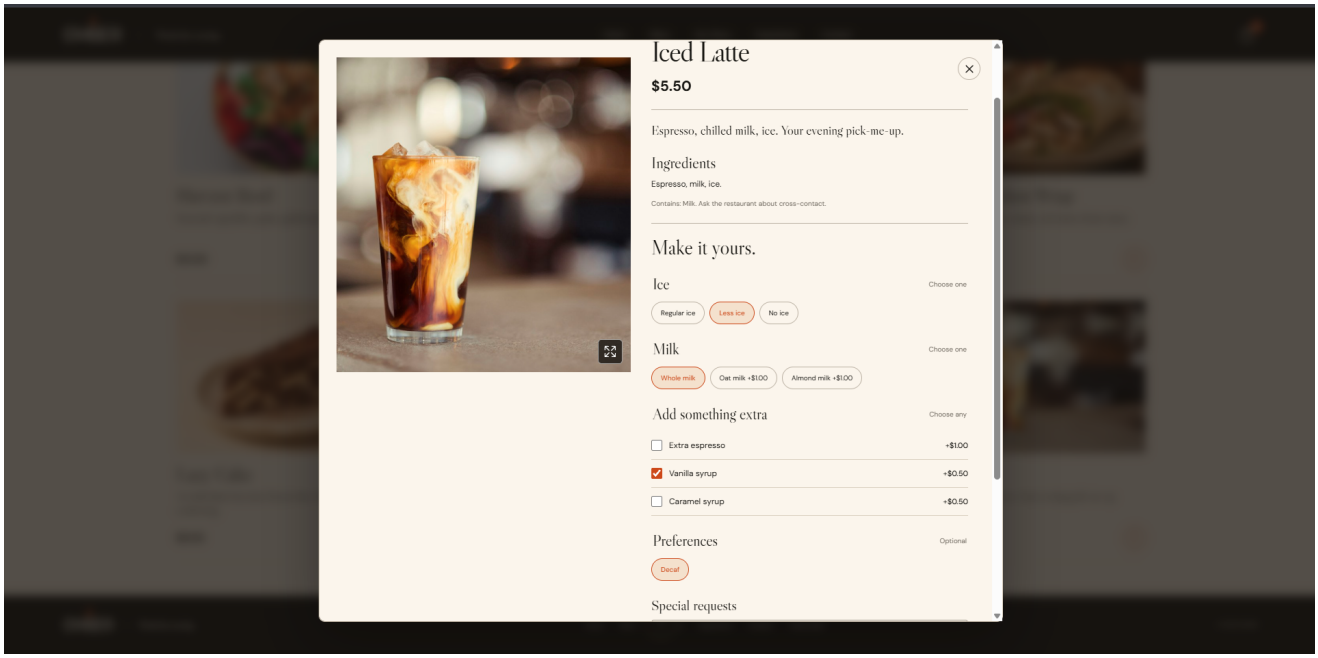


Figure 2. Iced Latte: selected milk, ice, extras and preferences.

03 / INTERFACE CONTINUED

Bag and order tracking.

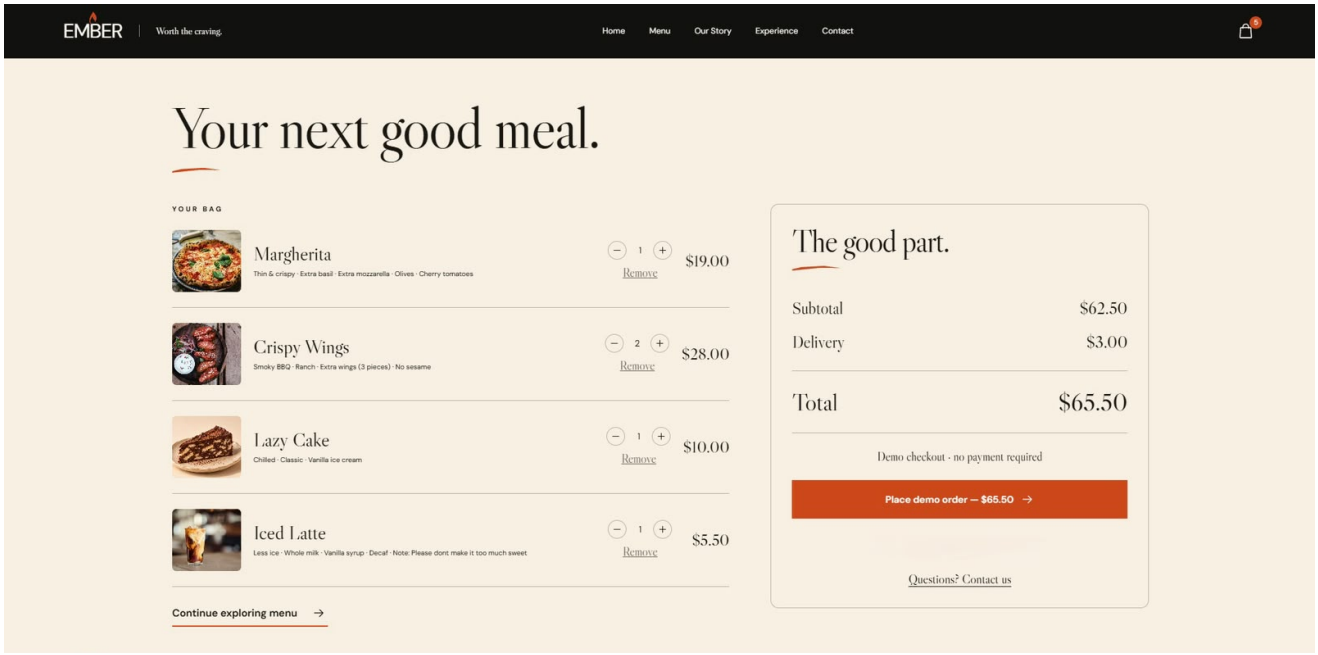


Figure 3. Bag and checkout: customized products, quantities and calculated totals.

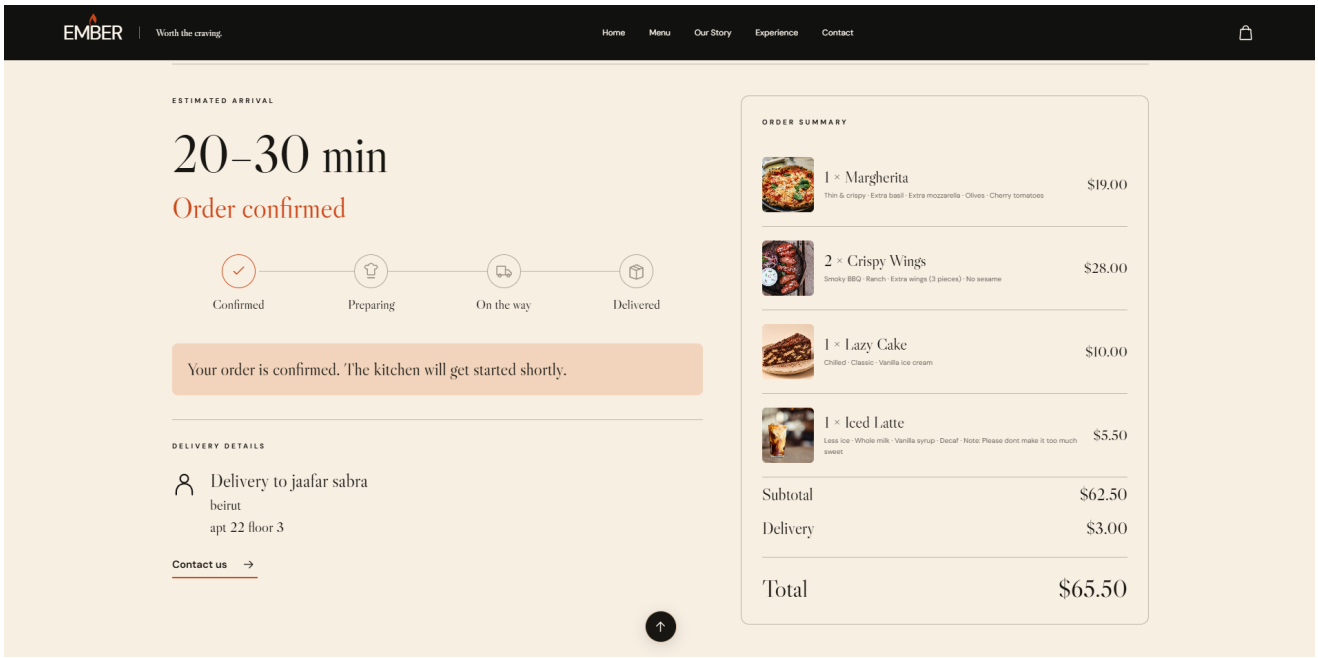


Figure 4. Confirmed delivery order with timeline, details and order summary.

03 / INTERFACE CONTINUED

Home and Our Story.

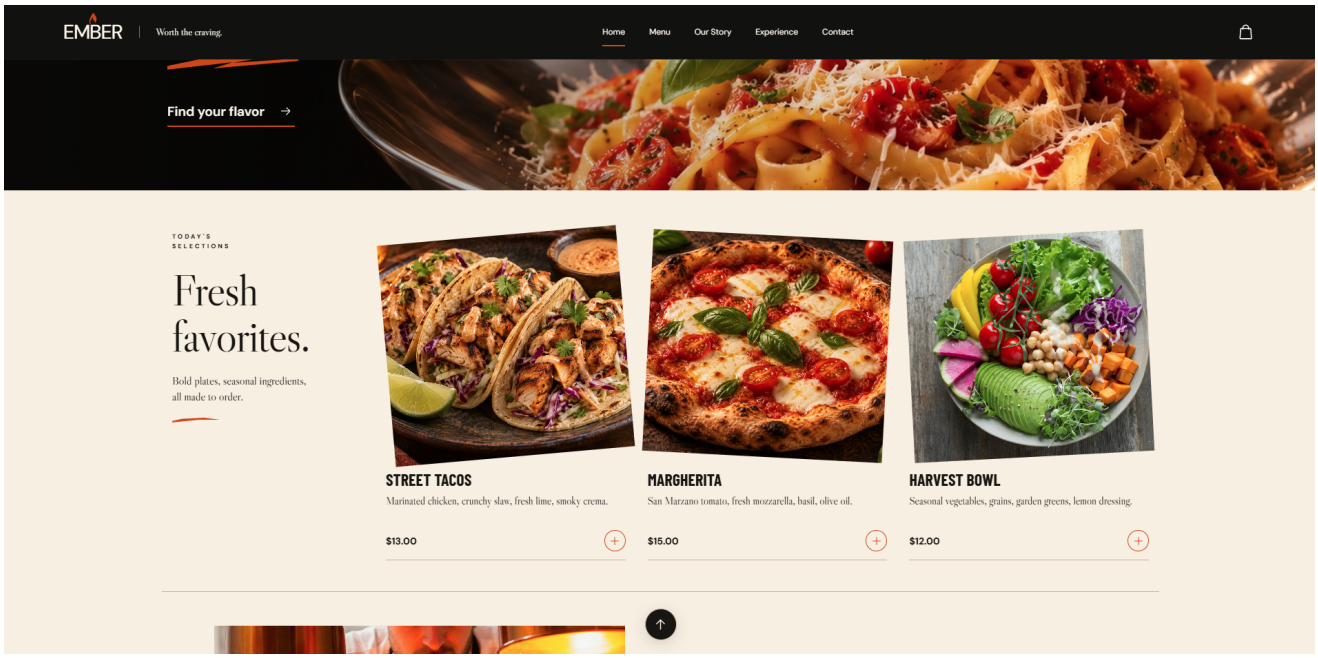


Figure 5. Home: curated favorites and kitchen introduction.

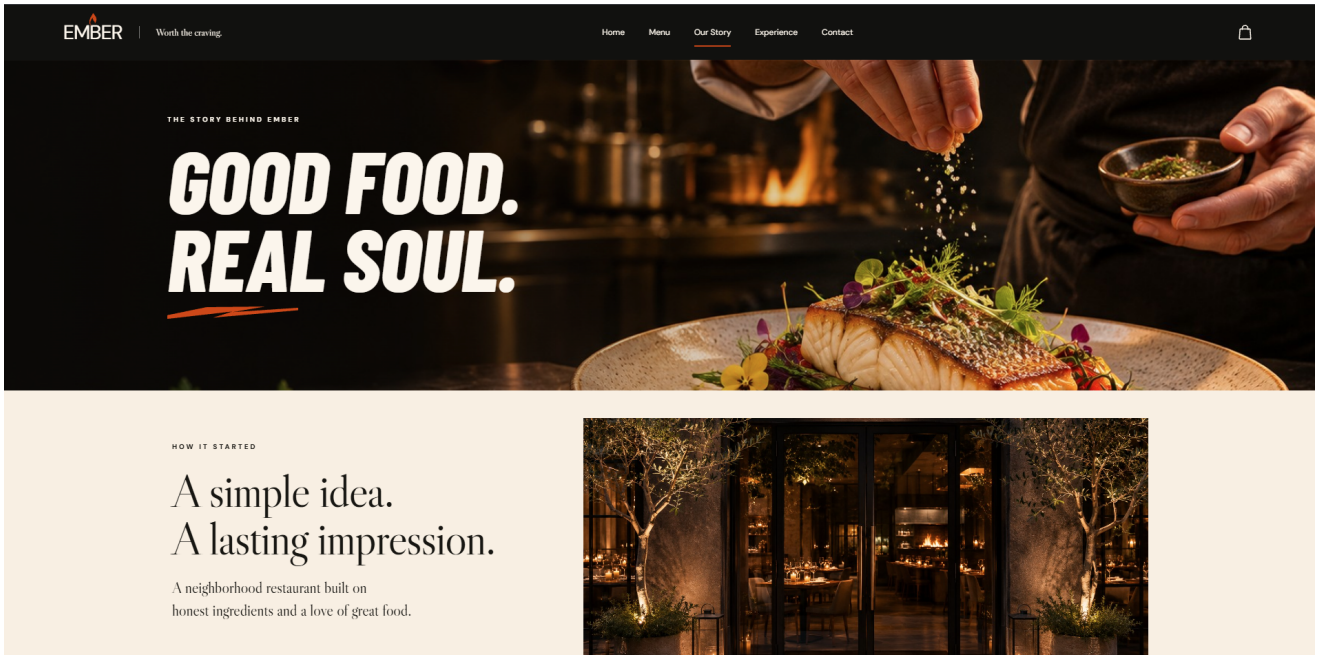


Figure 6. Our Story: hero and restaurant introduction.

03 / INTERFACE CONTINUED

Experience and Contact.

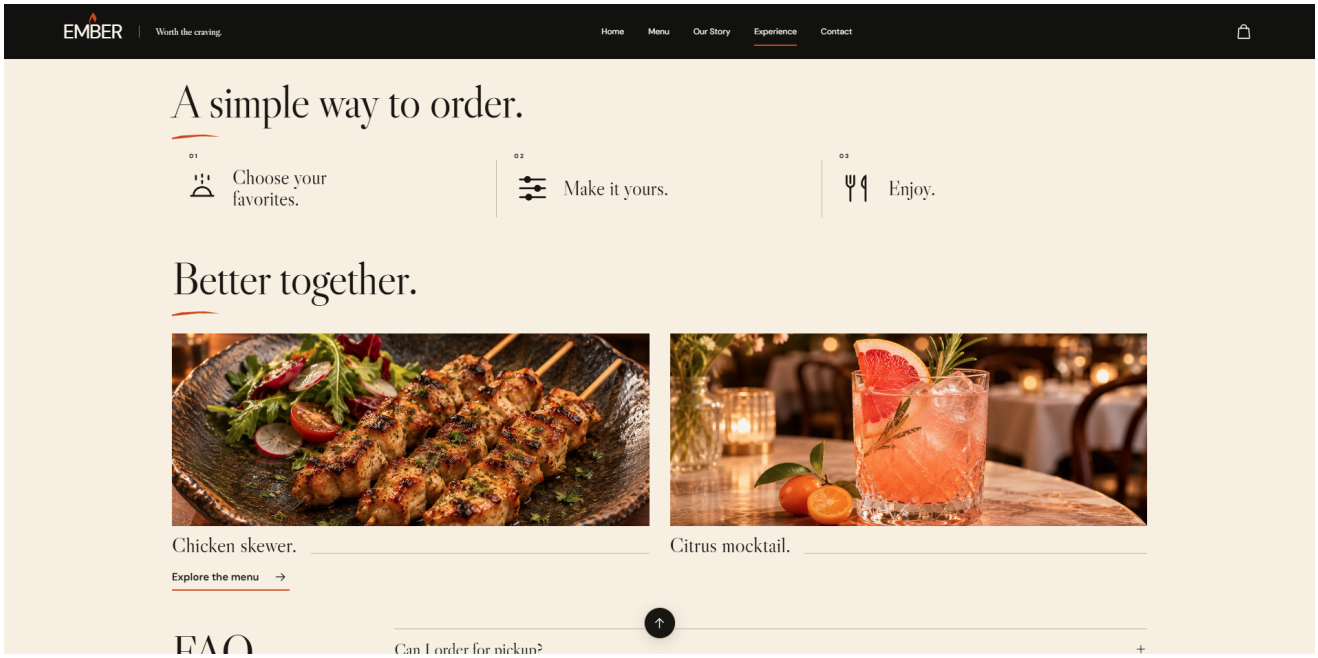


Figure 7. Experience: ordering steps and food-and-drink pairing.

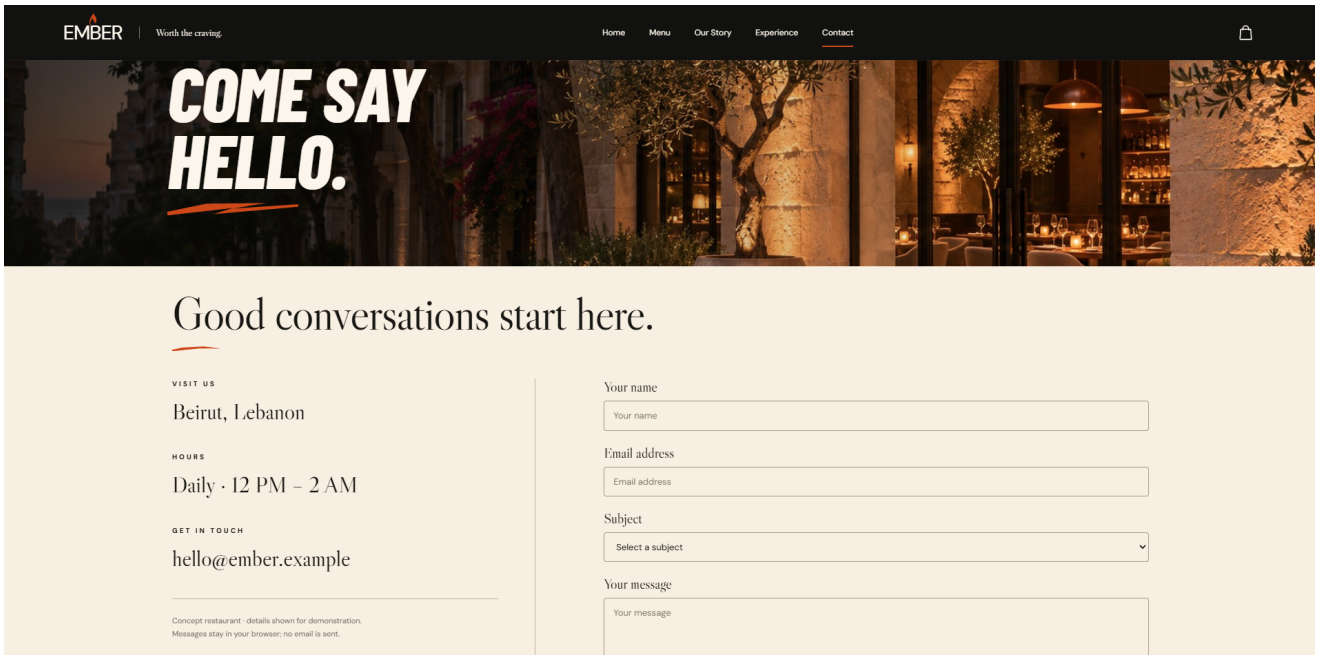


Figure 8. Contact: hours, contact details and labeled form.

The layout adapts at mobile breakpoints. Current phone screenshots and physical-device checks remain to finish. The enlarged-photo viewer, product flight and dialog transitions respect reduced-motion preferences.

04 / KEY CODE EXCERPTS

State turns choices into behavior.

Combined category and text filtering

The menu uses both constraints together. Search is case-insensitive and checks the name and description. Source: src/pages/Menu.jsx.

```
filtered = menu.filter(
  (x) =>
    (cat === "All" || x.category === cat) &&
    (x.name + " " + x.description)
      .toLowerCase()
      .includes(search.trim().toLowerCase()),
);
```

Paid choices and add-ons

Customization prices are recalculated from selected choices rather than stored as an independent total. Source: src/App.jsx.

```
const extraPrice = config
  ? config.groups.reduce(
      (sum, g) => sum +
        (g.options.find(o => o.label === choices[g.name])?.price || 0),
      0,
    ) + config.extras
      .filter(x => extras.includes(x.label))
      .reduce((sum, x) => sum + x.price, 0)
  : 0;
```

Line and order calculations

Unit price = base price + paid choices + extras. Line total = unit price x quantity. Subtotal sums the cart lines. Delivery adds $3; pickup adds no fee. The UI formats the calculated values to two decimal places.

Stored order snapshot

On successful frontend validation, checkout records the customer-entered details, mode, timestamp, totals and current item list. It persists that snapshot, then clears the bag and opens Tracking. The saved order is not altered when the next bag is edited.

05 / VALIDATION AND LIMITS

Evidence, not assumptions.

Check	Observed result
Production build	npm run build succeeds after source separation; 35 modules transformed.
Category and search controls	Browser interactions show matching items and empty-result behavior.
Customization pricing	Two $14 burgers with one $4 add-on each show $36 before delivery.
Multiple extras and notes	Selections and note retained in bag; two $19.50 units show $39.
Pickup / delivery	Earlier browser check: $18 subtotal becomes $21 with delivery and returns to $18 for pickup.
Photo viewer	Zoom controls and return-to-customization work.
Checkout and tracking	Validated demo order opens saved summary and simulated progress.
Contact and map	Local confirmation shown; real Beirut city map loads. Exact address is not claimed.
Motion / scroll control	Flying thumbnail is present and on-screen; scroll-to-top returns to zero.
After code cleanup	Build, menu, product customization, calculated price and checkout rechecked.
Pending final checks	Physical phone screenshots, cross-browser QA and final deployed-host smoke test.

Scope boundaries

No real order, payment, message or reservation is sent. Map embedding and Google Fonts require internet access; photographs are served locally. Contact details and menu descriptions are sample content. Allergen information is illustrative and must not be used for real food-safety decisions.

06 / SETUP AND HANDOFF

Reproducible and ready to publish.

Local setup

```
npm install
npm run dev
npm run build
```

Node.js 20+ is recommended. The build emits dist/. The repository includes vercel.json with the correct build command and output directory. No Phase 1 environment variables are required.

Submission links

Link	Status
GitHub repository	TO COMPLETE - account connection and repository publication pending.
Vercel	TO COMPLETE - instructor-approved live deployment pending.
Private review preview	https://orbit-kitchen-awada.hasanawadaa94193744.chatgpt.site

Assets and assistance

Food and editorial photographs combine original OpenAI-generated images with Unsplash photographs, served from public/assets/. The EMBER flame wordmark and UI icons are authored in source. Google Fonts supplies Barlow Condensed, DM Sans and Libre Caslon Display, with system fallbacks. AI assistance was used for design, implementation and documentation; students must follow course disclosure rules and understand the submitted code.

Before final submission

1. Verify the student and instructor details on the cover.
2. Add final GitHub and approved-host links.
3. Review the report against the final code and add current phone screenshots.
4. Complete the official individual contribution form if working in a group.
5. Submit the PDF report and GitHub link in Google Classroom by November 11, 3:00 PM.
6. Rehearse the presentation for Week 6, lecture 2.

Draft note

This report records the current frontend implementation and the checks observed during development. It does not assert GitHub or approved-host publication, final mobile certification, or individual student contributions.